

Factory Safety Incident Tracker

Week 2 Deliverable — Agile Planning and DevOps Workflow

1. User Stories & Acceptance Criteria

#	User Story	Acceptance Criteria
US-1	As a worker, I want to log a safety incident with title, description, severity, and location, so that it's recorded immediately.	Given valid inputs, when I submit the form, then the incident is saved and a confirmation is shown. Given a missing required field, submission is blocked with a clear error.
US-2	As a worker, I want the entry form to take under 2 minutes to fill, so that reporting doesn't disrupt my work.	Form has no more than 6 fields; severity and location use dropdowns, not free text.
US-3	As a safety officer, I want to search incidents by date range, so that I can review recent activity.	Given a date range filter, results only show incidents within that range.
US-4	As a safety officer, I want to filter incidents by status and severity, so that I can prioritize triage.	Filters can be combined (status=Open AND severity=High) and results update without a full page reload.
US-5	As a safety officer, I want to filter incidents by location, so that I can spot patterns at a specific site/zone.	Location filter lists all distinct locations from existing incidents.
US-6	As a plant manager, I want to see summary counts (open, closed, high-severity), so that I get a quick risk snapshot.	Dashboard shows 3 summary cards with live counts matching the database state.
US-7	As a plant manager, I want to click a summary card and see the matching incidents, so that I can drill into detail.	Clicking "Open" navigates to the dashboard pre-filtered to status=Open.
US-8	As a safety officer, I want to open a single incident's detail view, so that I can see full context before acting.	Detail view shows all fields plus reported timestamp and current status.
US-9	As a safety officer, I want to update an incident's status (Open to In Progress to Closed), so that resolution is tracked.	Status change is persisted and reflected immediately in the dashboard and summary counts.
US-10	As a plant manager, I want an alert/exception view of unresolved high-severity incidents, so that nothing critical slips through.	View lists all incidents where severity=High and status is not Closed, sorted oldest-first.
US-11	As a plant manager, I want overdue incidents (open more than X days) flagged in the alert view, so that I can escalate.	Incidents open longer than a configurable threshold (e.g. 3 days) are visually flagged.
US-12	As any user, I want the dashboard to load in under 3 seconds, so that the tool doesn't slow me down.	Page load time measured under normal load is under 3s for up to 500 incidents.

2. Product Backlog (MoSCoW-Prioritized)

Priority	Backlog Items
Must Have	US-1, US-2, US-3, US-4, US-6, US-7, US-8, US-9, US-10
Should Have	US-5, US-11
Could Have	US-12 (performance target - validated continuously, not a single ticket)
Won't Have (MVP)	Auth/RBAC, notifications, multi-site support, exports - deferred to future enhancements

3. Kanban / Task Board

Columns: Backlog → To Do → In Progress → Review → Done

Tool: GitHub Projects, linked directly to the repository created in Week 4, so issues and pull requests automatically sync to the board.

4. 15-Week Sprint Plan (Backlog → Week Mapping)

Sprint / Weeks	Focus	Backlog Items Covered
Sprint 1 (Wk 1-2)	Planning	Planning artefacts only
Sprint 2 (Wk 3-4)	Architecture + repo setup	Foundation, no stories yet
Sprint 3 (Wk 5)	Feature 1	US-1, US-2
Sprint 4 (Wk 6)	Remaining MVP features	US-3 to US-11
Sprint 5 (Wk 7-8)	CI/CD foundation	Enables US-12 validation via pipeline
Sprint 6 (Wk 9-10)	Quality gate	Test coverage for US-1 to US-11
Sprint 7 (Wk 11-12)	Containerization	Non-functional, infrastructure
Sprint 8 (Wk 13-14)	Provisioning	Non-functional, infrastructure
Sprint 9 (Wk 15)	Release	Full backlog validated end-to-end

5. Definition of Done

An item is considered Done only when:

- Code is committed to a feature branch with meaningful commit messages
- A Pull Request has been opened, reviewed, and approved
- The change builds successfully (mvn clean package)
- Relevant unit/Selenium tests pass
- The feature is merged into develop
- The feature is deployed and manually verified in the dev/staging environment

- Acceptance criteria for the linked user story are met
- Documentation (README/docs) is updated if behavior changed

6. DevOps Lifecycle Diagram

The diagram below shows the end-to-end DevOps lifecycle for the Factory Safety Incident Tracker: Plan and Code feed a Jenkins Build and Selenium Test stage; passing builds are Released as a Docker image and Deployed via Ansible/Puppet; the app then runs in Operate, and Monitor findings loop back into Plan for the next iteration.

